

Chapter 8 Algorithms

After studying this chapter, the student should be able to:

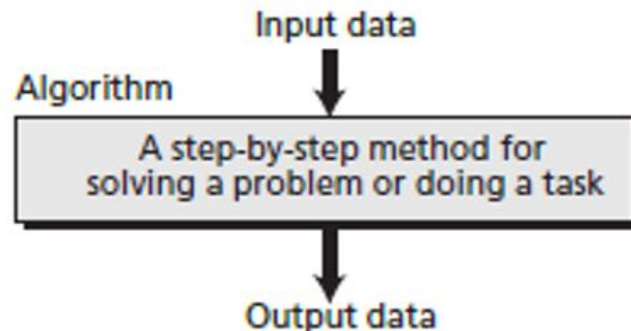
- Define an algorithm and relate it to problem solving.
- Define three constructs – sequence, selection and repetition – and describe their use in algorithms.
- Describe UML diagrams and how they can be used when representing algorithms.
- Describe pseudocode and how it can be used when representing algorithms.
- List basic algorithms and their applications.
- Describe the concept of sorting and understand the mechanisms behind three primitive sorting algorithms.
- Describe the concept of searching and understand the mechanisms behind two common searching algorithms.
- Define subalgorithms and their relation to algorithms.
- Distinguish between iterative and recursive algorithms.

8.1 CONCEPT

8.1.1 Informal definition

- An informal definition of an algorithm is: **a step-by-step method for solving a problem or doing a task.**
- In this definition, an algorithm is independent of the computer system. More specially, we should also note that the algorithm accepts **input data** and creates **output data**.

Figure 8.1 *Informal definition of an algorithm used in a computer*

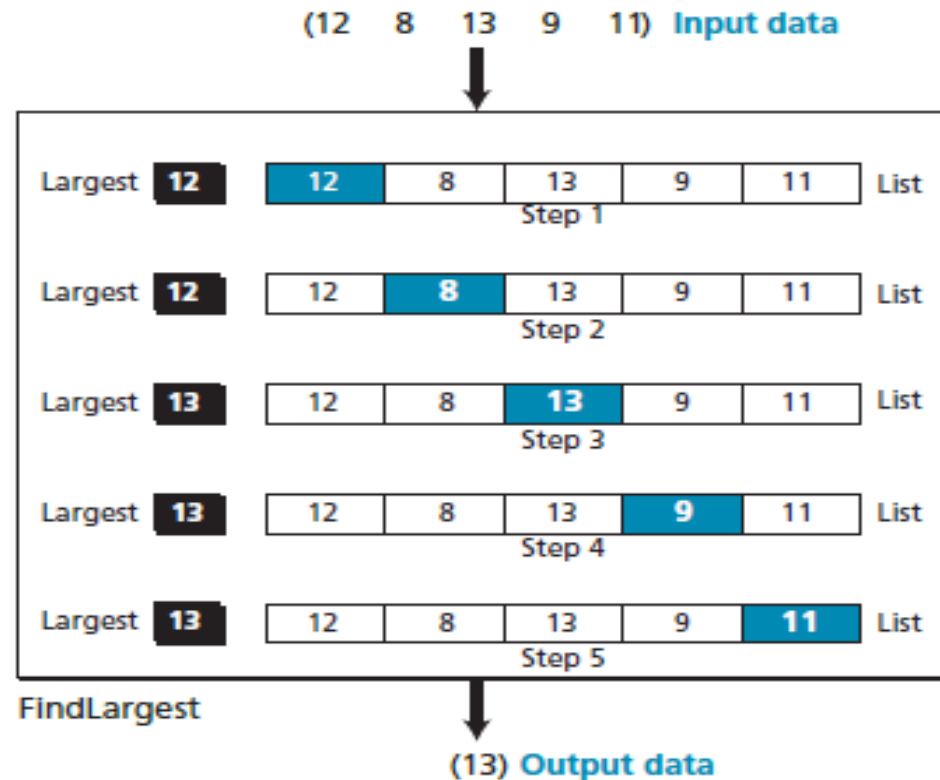


Example

- We want to develop an algorithm for finding the largest integer among a list of positive integers.
 - The algorithm should find the largest integer among a list of any size (for example 5, 1000, 10 000, 1 000 000).
 - The algorithm should be general and not depend on the number of integers.
 - It is obvious that finding the largest integer among many integers is a task that cannot be done in one step, either by a human or a computer.
 - The algorithm needs to test each integer one by one.
- To solve this problem, we need an intuitive approach.
 - First use a small number of integers (for example, five), then extend the solution to any number of integers.
 - Our solution for five integers follows the same principles and restrictions for one thousand or one million integers.
 - Assume, even for a five-integer case, that the algorithm handles the integers one by one. It looks at the first integer without knowing the values of the remaining integers. After it handles the first one, it looks at the second integer, and so on.
 - Figure 8.2 shows one way to solve this problem.

- We call the algorithm FindLargest.
 - Each algorithm has a name to distinguish it from other algorithms.
 - The algorithm receives a list of five integers as input and gives the largest integer as output.

Figure 8.2 *Finding the largest integer among five integers*



Input

The algorithm accepts the list of five integers as input.

Processing

The algorithm uses the five steps to find the largest integer.

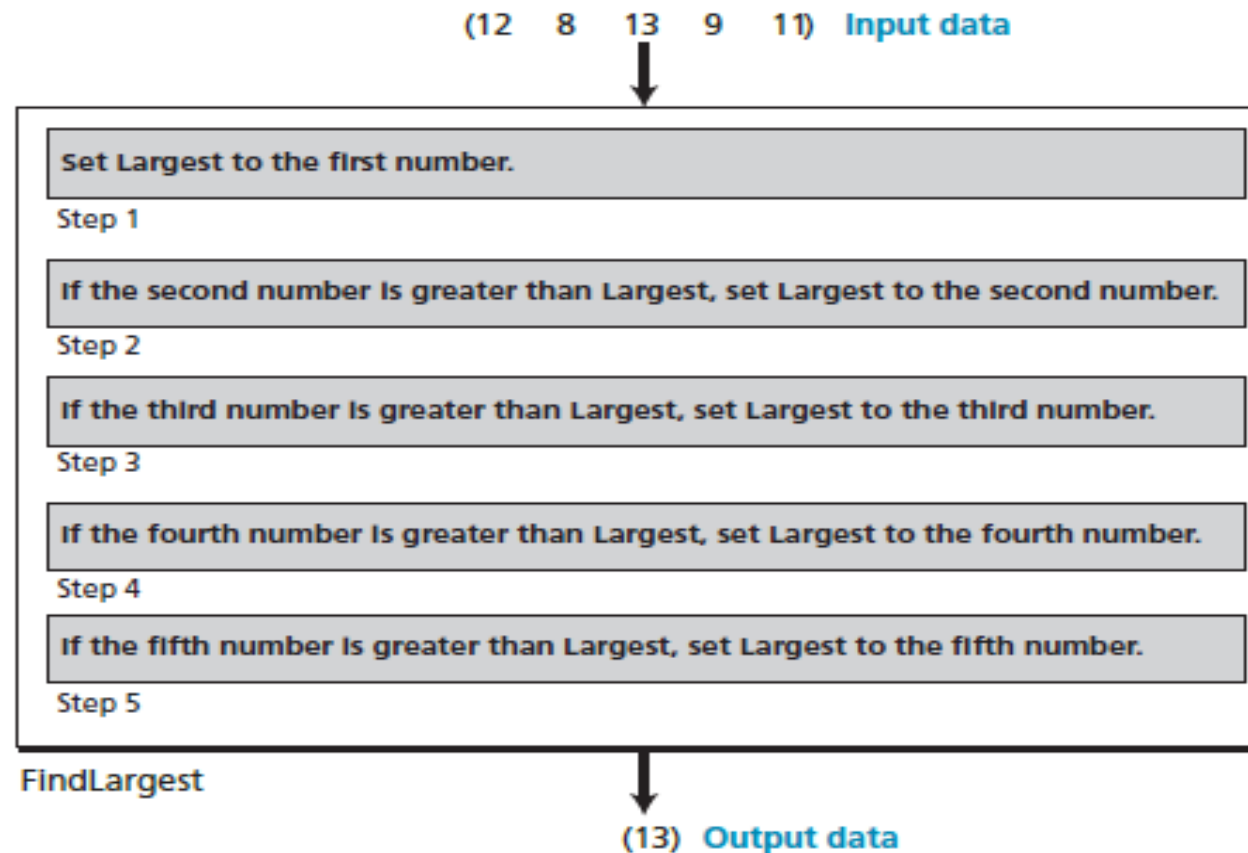
Output

Because there are no more integers to be processed, the algorithm outputs the value of Largest, which is 13.

8.1.2 Defining actions

Figure 8.2 does not show what should be done in each step. We can modify the figure to show more details.

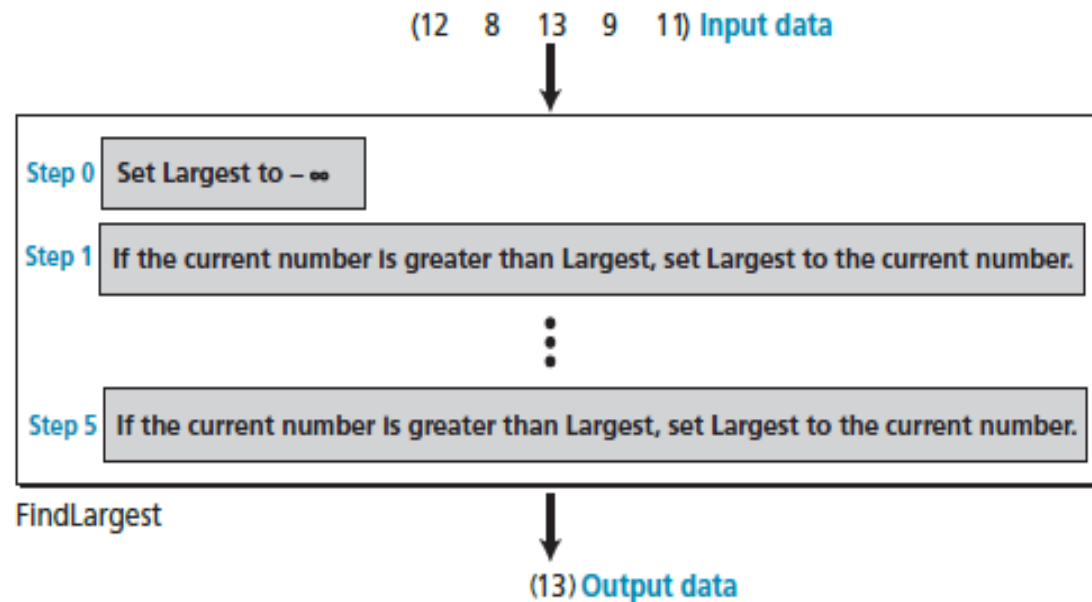
Figure 8.3 *Defining actions in FindLargest algorithm*



8.1.3 Refinement

- This algorithm needs refinement to be acceptable to the programming community. There are two problems.
 - First, the action in the first step is different than those for the other steps.
 - Second, the wording is not the same in steps 2 to 5.

Figure 8.4 *FindLargest refined*

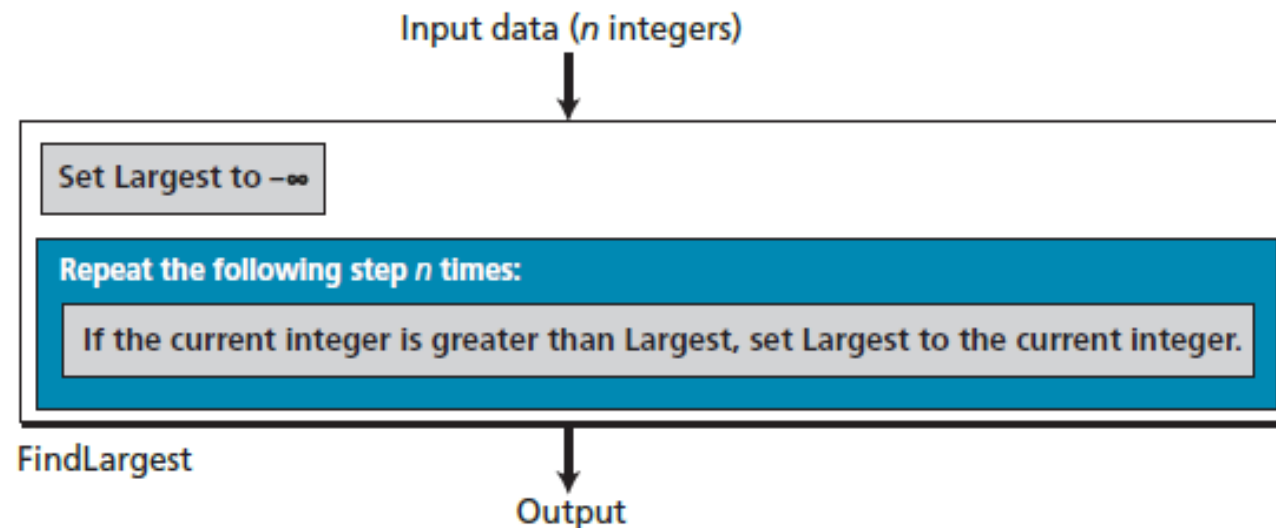


- We can easily redefine the algorithm to remove these two inconveniences by changing the wording in steps 2 to 5 to “If the current integer is greater than Largest, set Largest to the current integer.”
- The reason that the first step is different than the other steps is because Largest is not initialized. If we initialize Largest to $-\infty$ (minus infinity), then the first step can be the same as the other steps, so we add a new step, calling it step 0 to show that it should be done before processing any integers.

8.1.4 Generalization

Is it possible to generalize the algorithm?

Figure 8.5 *Generalization of FindLargest*

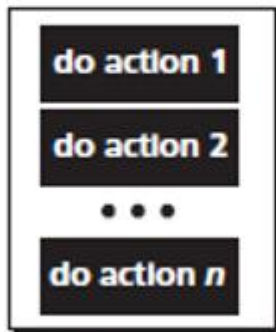


- We want to find the largest of n positive integers, where n can be 1000, 1,000,000, or more.
- Of course, we can follow Figure 8.4 and repeat each step. But if we change the algorithm to a program, then we need to actually type the actions for n steps!
- There is a better way to do this. We can tell the computer to repeat the steps n times. We now include this feature in our pictorial algorithm (Figure 8.5).

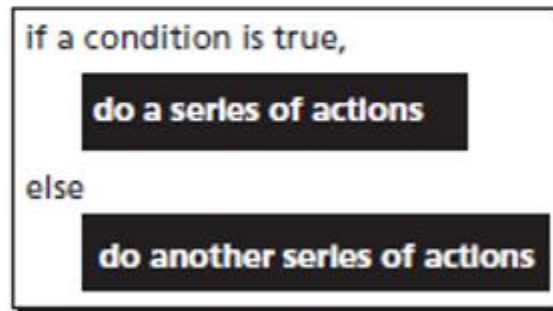
8.2 THREE CONSTRUCTS

- Computer scientists have defined three constructs for a structured program or algorithm.
- The idea is that a program must be made of a combination of only these three constructs: **sequence**, **decision** (selection), and **repetition** (Figure 8.6).
- It has been proven there is no need for any other constructs. Using only these constructs makes a program or an algorithm easy to understand, debug, or change.

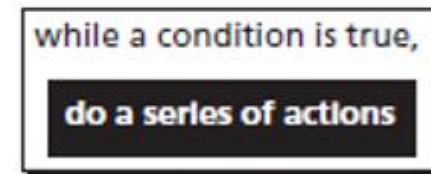
Figure 8.6 *Three constructs*



a. Sequence



b. Decision



c. Repetition

8.2.1 Sequence

- The first construct is called the sequence. An algorithm, and eventually a program, is a sequence of instructions, which can be a simple instruction or either of the other two constructs.

8.2.2 Decision

- Some problems cannot be solved with only a sequence of simple instructions. Sometimes we need to test a condition. If the result of testing is true, we follow a sequence of instructions: if it is false, we follow a different sequence of instructions. This is called the decision (selection) construct.

8.2.3 Repetition

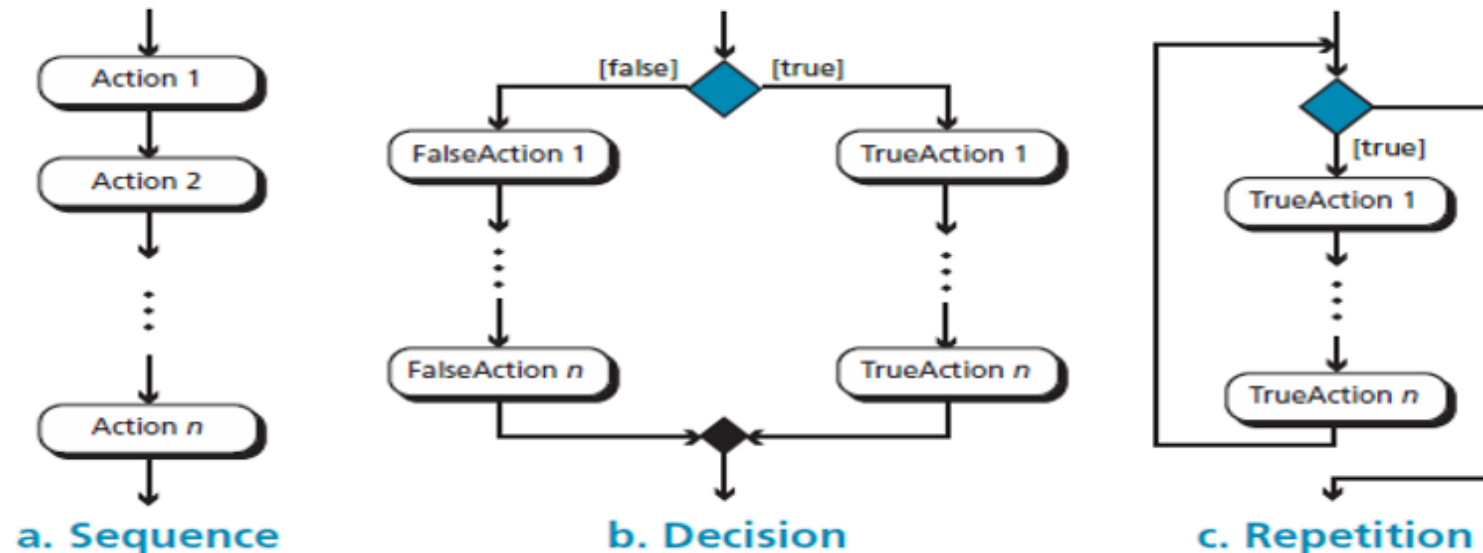
- In some problems, the same sequence of instructions must be repeated. We handle this with the repetition or ***loop*** construct. Finding the largest integer among a set of integers can use a construct of this kind.

ALGORITHM REPRESENTATION – UML and PSEUDOCODE

8.3.1 UML

- Unified Modeling Language (UML) is a pictorial representation of an algorithm. It hides all the details of an algorithm in an attempt to give the “big picture” and to show how the algorithm flows from beginning to end.
- UML is covered in detail in Appendix B. Here we show only how the three constructs are represented using UML (Figure 8.7).

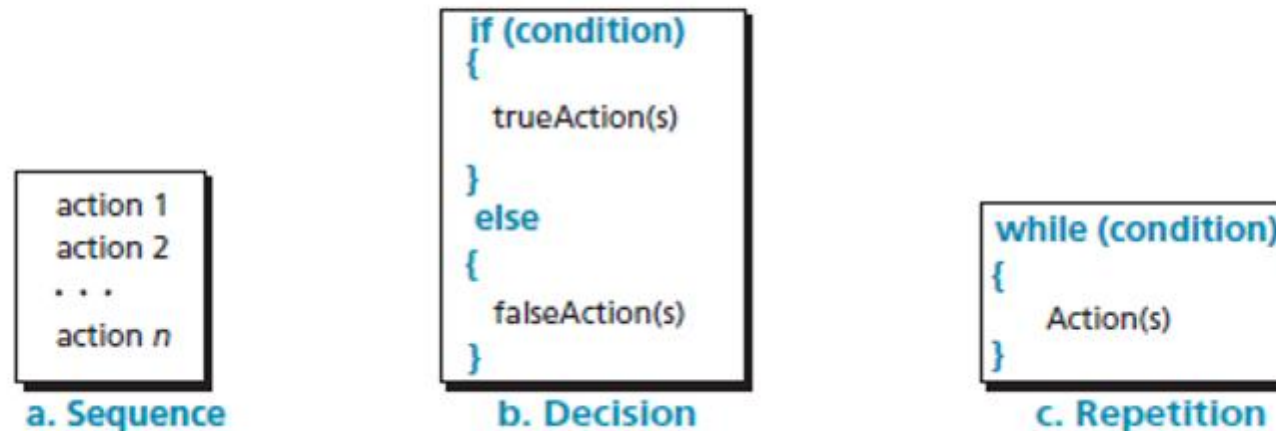
Figure 8.7 UML for three constructs



8.3.2 Pseudocode

- Pseudocode is an English-language-like representation of an algorithm.
- There is no standard for pseudocode—some people use a lot of detail, others use less. Some use a code that is close to English, while others use a syntax like the Pascal programming language.
- Pseudocode is covered in detail in Appendix C. Here we show only how the three constructs can be represented by pseudocode (Figure 8.8).

Figure 8.8 *Pseudocode for three constructs*



Algorithms

Example 8.1

Write an algorithm in pseudocode that finds the sum of two integers.

Algorithm 8.1 Calculating the sum of two integers

```
Algorithm: SumOfTwo (first, second)
Purpose: Find the sum of two integers
Pre: Given: two integers (first and second)
Post: None
Return: The sum value
{
    sum ← first + second
    return sum
}
```

Example 8.2

Write an algorithm to change a numeric grade to a pass/no pass grade.

Algorithm 8.2 Assigning pass/no pass grade

Algorithm: Pass/NoPass (score)

Purpose: Creates a pass/no pass grade given the score

Pre: Given: the score to be changed to grade

Post: None

Return: The grade

```
{  
    if (score  $\geq$  70)  
        grade  $\leftarrow$  "pass"  
    else  
        grade  $\leftarrow$  "nopass"  
    return grade  
}
```

Example 8.3

Write an algorithm to change a numeric grade to a letter grade.

Algorithm 8.3 Assigning a letter grade

Algorithm: LetterGrade (score)

Purpose: Find the letter grade corresponding to the given score

Pre: Given: a numeric score

Post: None

Return: A letter grade

```
{  
    if ( $100 \geq \text{score} \geq 90$ )  
        grade  $\leftarrow$  'A'  
    if ( $89 \geq \text{score} \geq 80$ )  
        grade  $\leftarrow$  'B'  
    if ( $79 \geq \text{score} \geq 70$ )  
        grade  $\leftarrow$  'C'  
    if ( $69 \geq \text{score} \geq 60$ )  
        grade  $\leftarrow$  'D'  
    if ( $59 \geq \text{score} \geq 0$ )  
        grade  $\leftarrow$  'F'  
    return grade  
}
```

Example 8.4

Write an algorithm to find the largest of a set of integers. We do not know the number of integers.

Algorithm 8.4 Finding the largest integer among a set of integers

Algorithm: FindLargest (list)

Purpose: Find the largest integer among a set of integers

Pre: Given: the set of integers

Post: None

Return: The largest integer

```
{  
    largest  $\leftarrow -\infty$   
    while (more integers to check)  
    {  
        current  $\leftarrow$  next integer  
        if (current > largest)  
            largest  $\leftarrow$  current  
    }  
    return largest  
}
```

Example 8.5

Write an algorithm to find the largest of the first 1000 integers in a set of integers.

Algorithm 8.5 Find the largest integers among 1000 integers

Algorithm: FindLargest (list)

Purpose: Find and return the largest integer among the first 1000 integers

Pre: Given the set of integers with more than 1000 integers

Post: None

Return: The largest integer

```
{
    largest  $\leftarrow +\infty$ 
    counter  $\leftarrow 1$ 
    while (counter  $\leq 1000$ )
    {
        current  $\leftarrow$  next integer
        if (current  $<$  largest)
            largest  $\leftarrow$  current
        counter  $\leftarrow$  counter + 1
    }
    return largest
}
```

8.4 A MORE FORMAL DEFINITION

Algorithm: An ordered set of unambiguous steps that produces a result and terminates in a finite time.

8.4.1 Well-Defined

- An algorithm must be a well-defined, ordered set of instructions.

8.4.2 Unambiguous steps

- Each step in an algorithm must be clearly and unambiguously defined. If one step is to add two integers, we must define both “integers” as well as the “add” operation: we cannot for example use the same symbol to mean addition in one place and multiplication somewhere else.

8.4.3 Produce a result

- An algorithm must produce a result, otherwise it is useless. The result can be data returned to the calling algorithm, or some other effect (for example, printing).

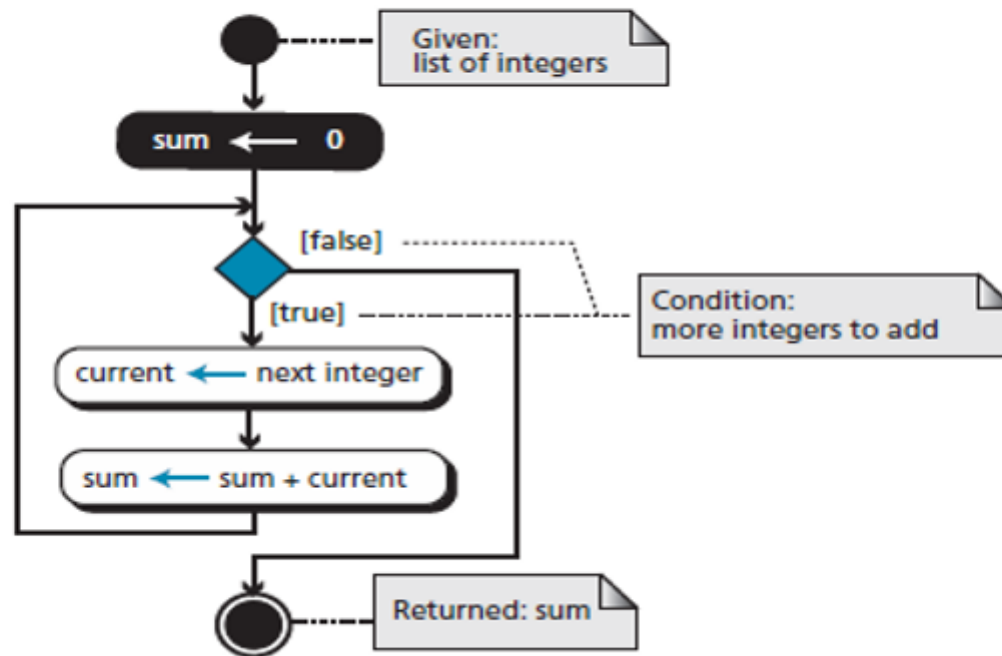
8.4.4 Terminate in a finite time

- An algorithm must terminate (halt). If it does not (that is, it has an infinite loop), we have not created an algorithm.
- In Chapter 17 we will discuss solvable and unsolvable problems, and we will see that a solvable problem has a solution in the form of an algorithm that terminates.

8.5 BASIC ALGORITHMS

8.5.1 Summation algorithm

Figure 8.9 *Summation algorithm*

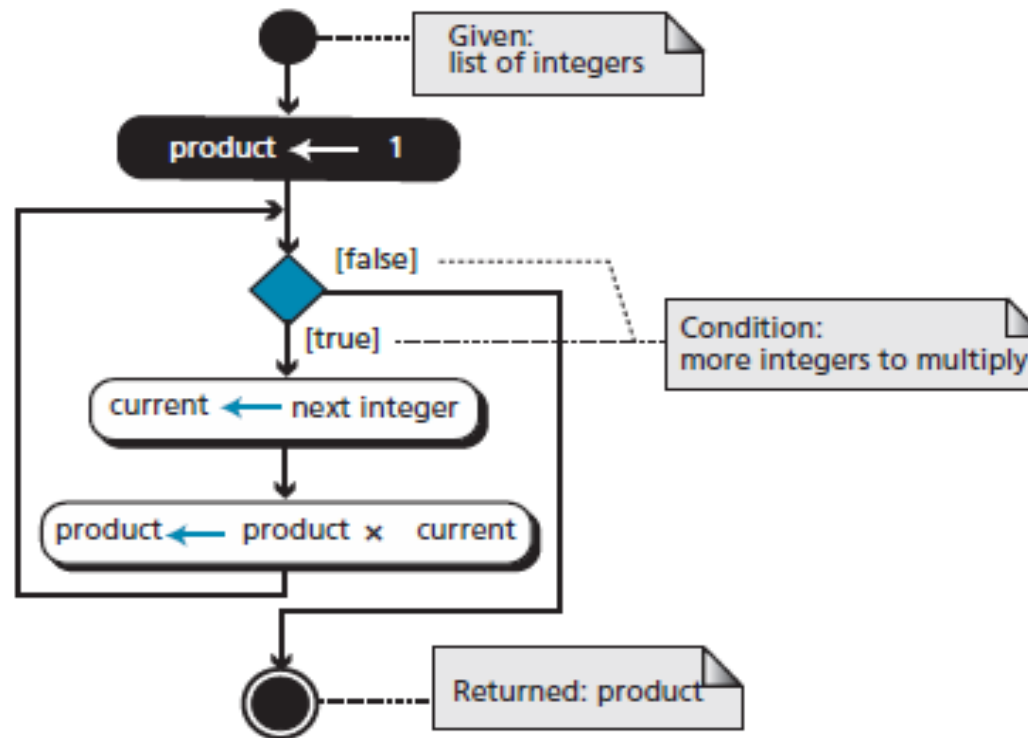


A summation algorithm has three logical parts:

1. Initialization of the sum at the beginning.
2. Return of the result after exiting from the loop.
3. The loop, which in each iteration adds a new integer to the sum.

8.5.2 Product algorithm

Figure 8.10 *Product algorithm*



A product algorithm has three logical parts:

1. Initialization of the product at the beginning.
2. The loop, which in each iteration multiplies a new integer with the product.
3. Return of the result after exiting from the loop.

8.5.4 Sorting

- One of the most common applications in computer science is sorting, which is the process by which data is arranged according to its values. People are surrounded by data. If the data was not ordered, it would take hours and hours to find a single piece of information. Imagine the difficulty of finding someone's telephone number in a telephone book that is not ordered.
- In this section, we introduce three sorting algorithms: **selection sort**, **bubble sort**, and **insertion sort**.
- These three sorting algorithms are the foundation of faster sorting algorithms used in computer science today.

Selection sorts

Figure 8.11 Selection sort

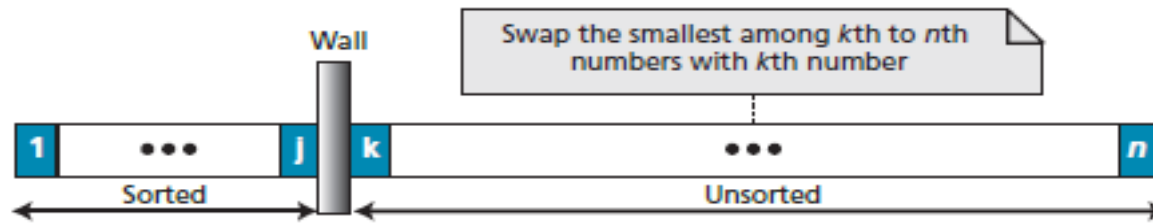
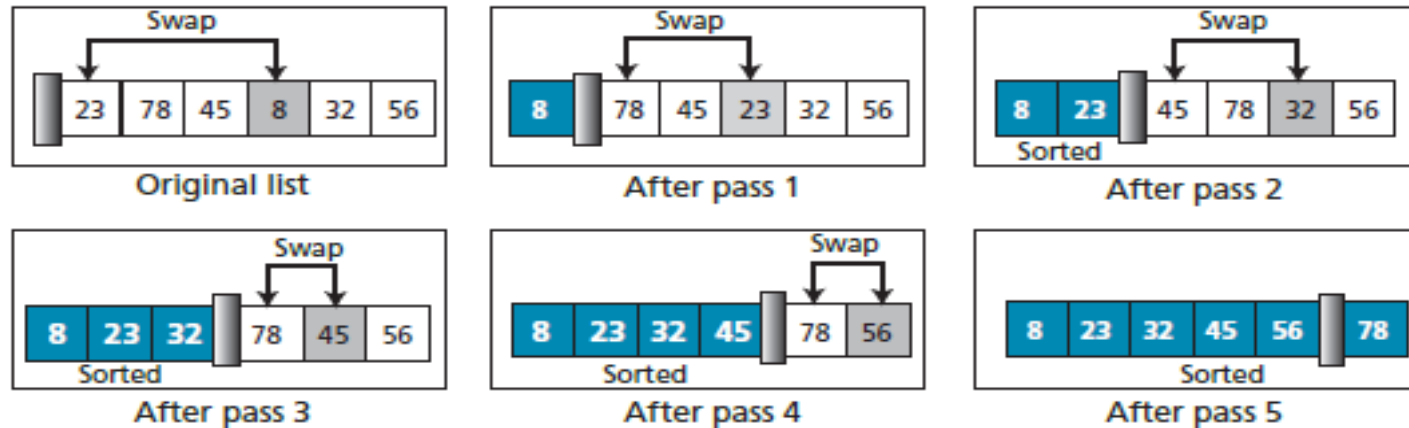


Figure 8.12 traces a set of six integers as we sort them.

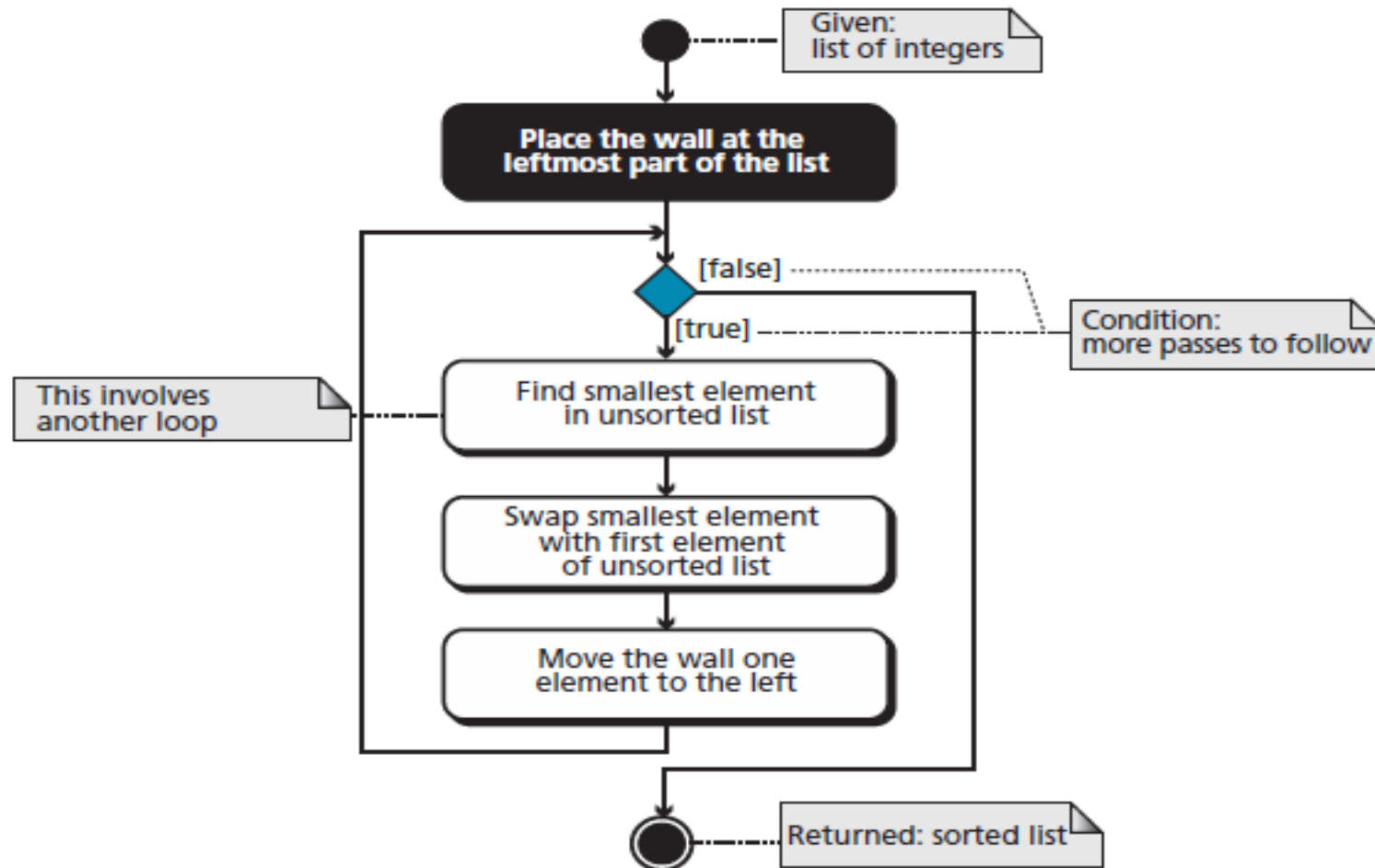
Figure 8.12 Example of selection sort



- In a selection sort, the list to be sorted is divided into two sublists — sorted and unsorted — which are separated by an imaginary wall.
- We find the smallest element from the unsorted sublist and swap it with the element at the beginning of the unsorted sublist.
- After each selection and swap, the imaginary wall between the two sublists moves one element ahead.

Selection sort algorithm

Figure 8.13 Selection sort algorithm



Bubble sort

Figure 8.14 *Bubble sort*

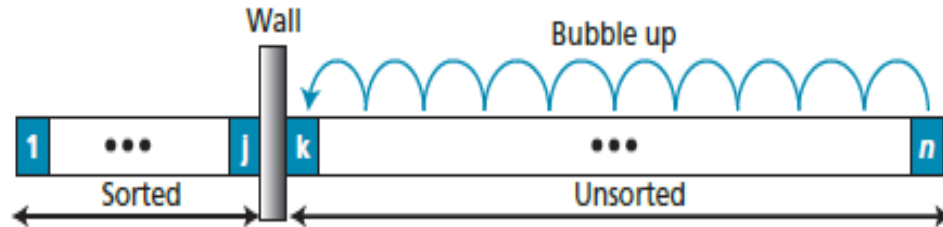
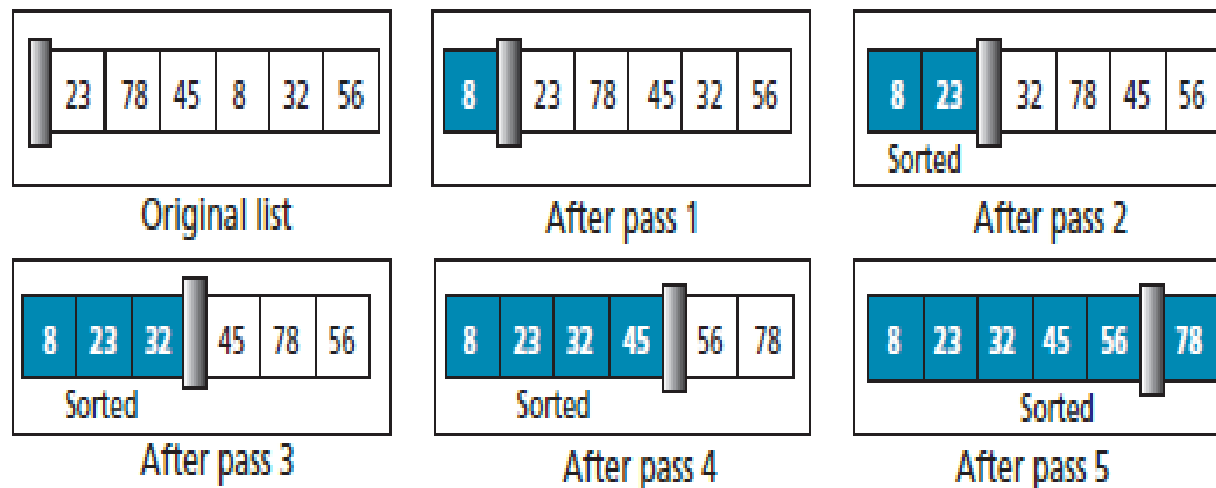


Figure 8.15 *Example of bubble sort*



- In the bubble sort method, the list to be sorted is also divided into two sublists — sorted and unsorted.
- The smallest element is bubbled up from the unsorted sublist and moved to the sorted sublist.
- After the smallest element has been moved to the sorted list, the wall moves one element ahead.

Insertion sort

Figure 8.16 *Insertion sort*

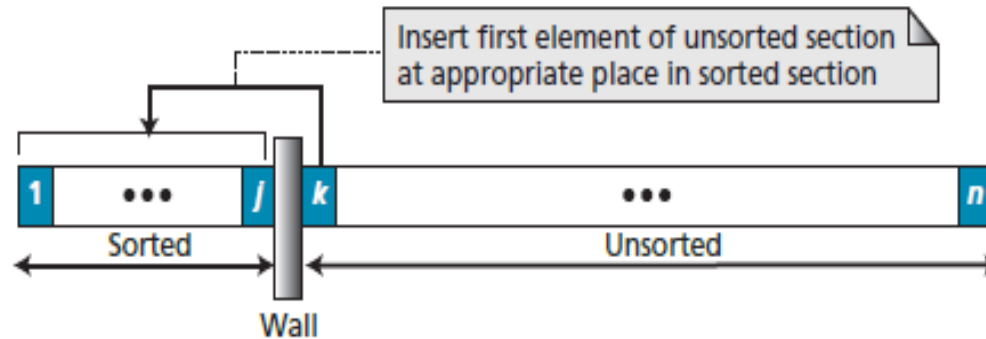
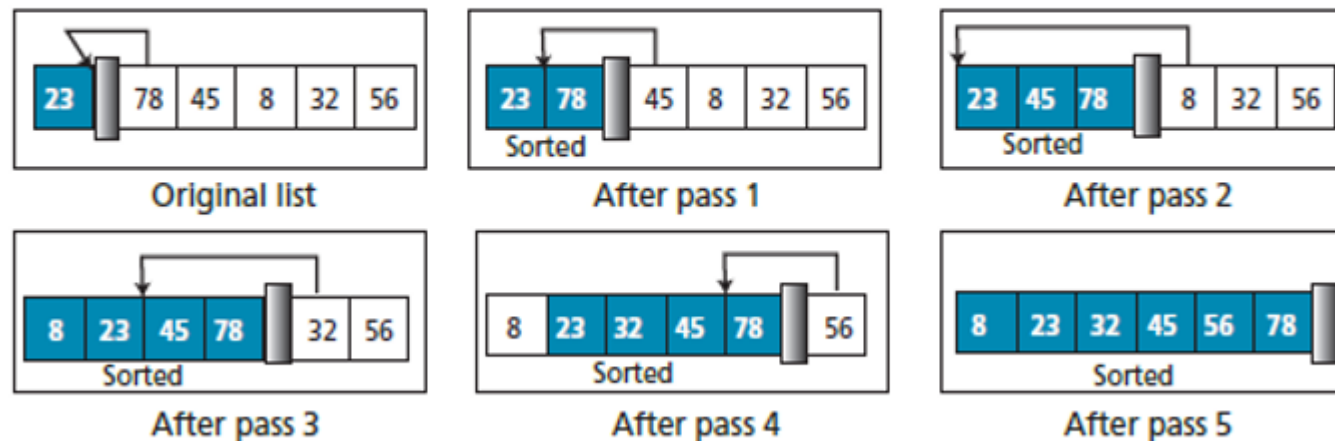


Figure 8.17 *Example of insertion sort*



- The insertion sort algorithm is one of the most common sorting techniques, and it is often used by card players.
- Each card a player picks up is inserted into the proper place in their hand of cards to maintain a particular sequence.

8.5.5 Searching

Sequential search

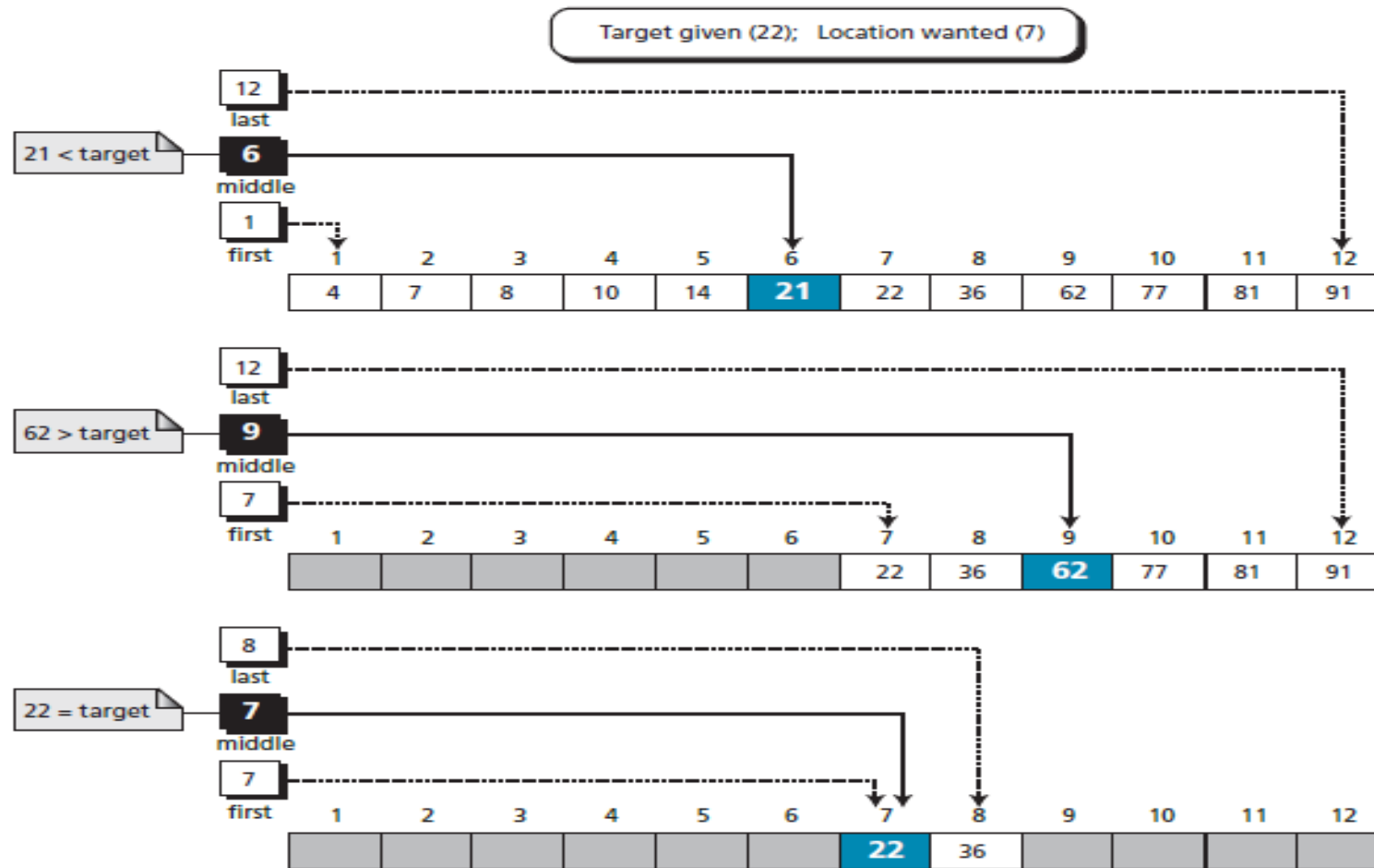
Figure 8.18 An example of a sequential search



- Sequential search is used if the list to be searched is not ordered. Generally, we use this technique only for small lists, or lists that are not searched often. In other cases, the best approach is to first sort the list and then search it using the binary search discussed later.
- In a sequential search, we start searching for the target from the beginning of the list. We continue until we either find the target or reach the end of the list.

Binary search

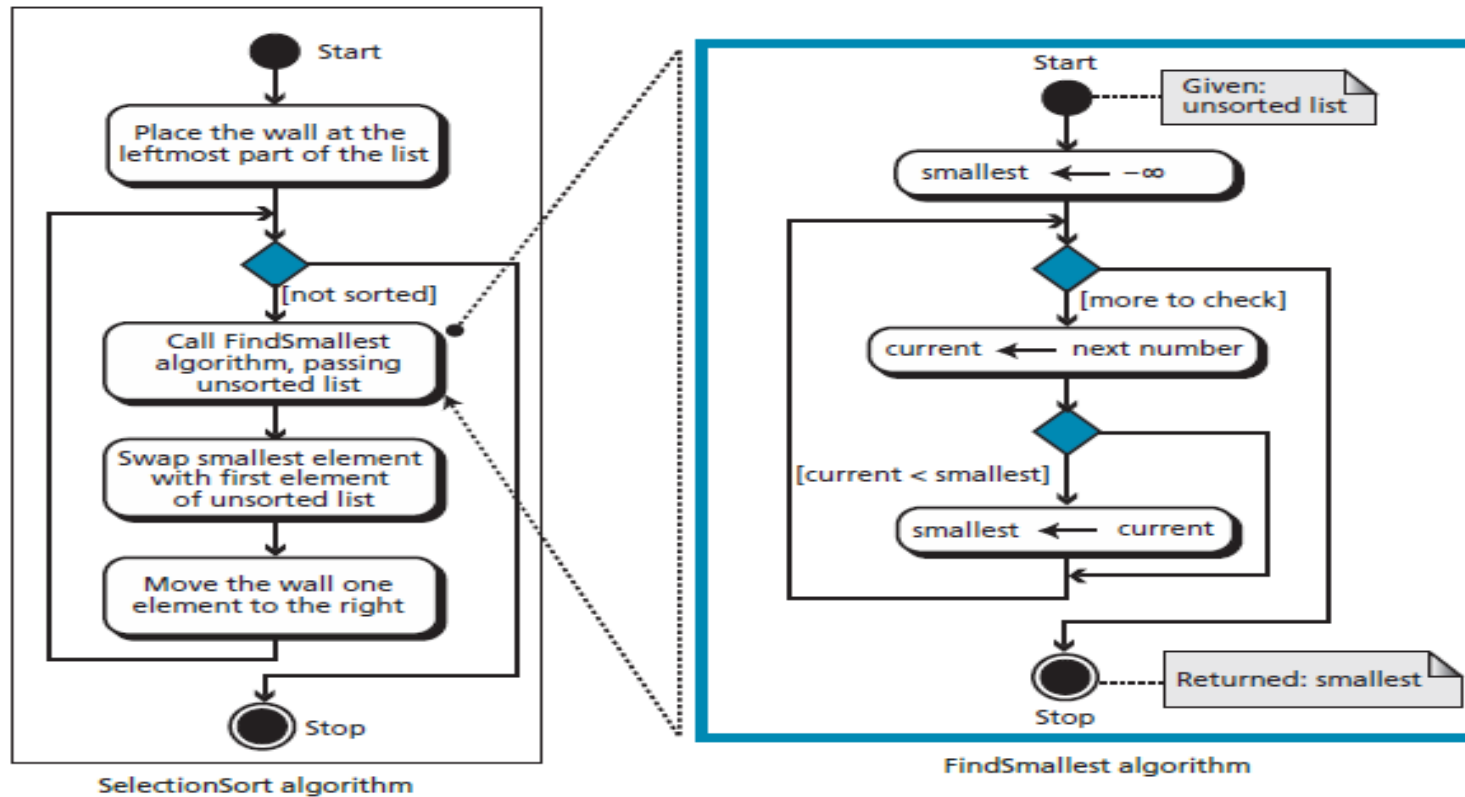
Figure 8.19 Example of a binary search



- The sequential search algorithm is very slow. If we have a list of a million elements, we must do a million comparisons in the worst case. If the list is not sorted, this is the only solution. If the list is sorted, however, we can use a more efficient algorithm called binary search. Generally speaking, programmers use a binary search when a list is large.
- A binary search starts by testing the data in the element at the middle of the list. This determines whether the target is in the first half or the second half of the list. If it is in the first half, there is no need to further check the second half. If it is in the second half, there is no need to further check the first half. In other words, we eliminate half the list from further consideration.

SUBALGORITHMS

Figure 8.20 Concept of a subalgorithm



- The three programming constructs described in Section 8.2 allow us to create an algorithm for any solvable problem.
- The principles of structured programming, however, require that an algorithm be broken into small units called subalgorithms.
- Each subalgorithm is in turn divided into smaller subalgorithms.
- A good example is the algorithm for the selection sort in Figure 8.13.

RECURSION

Figure 8.21 *Iterative definition of factorial*

$$\text{Factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1) \times (n-2) \cdots 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

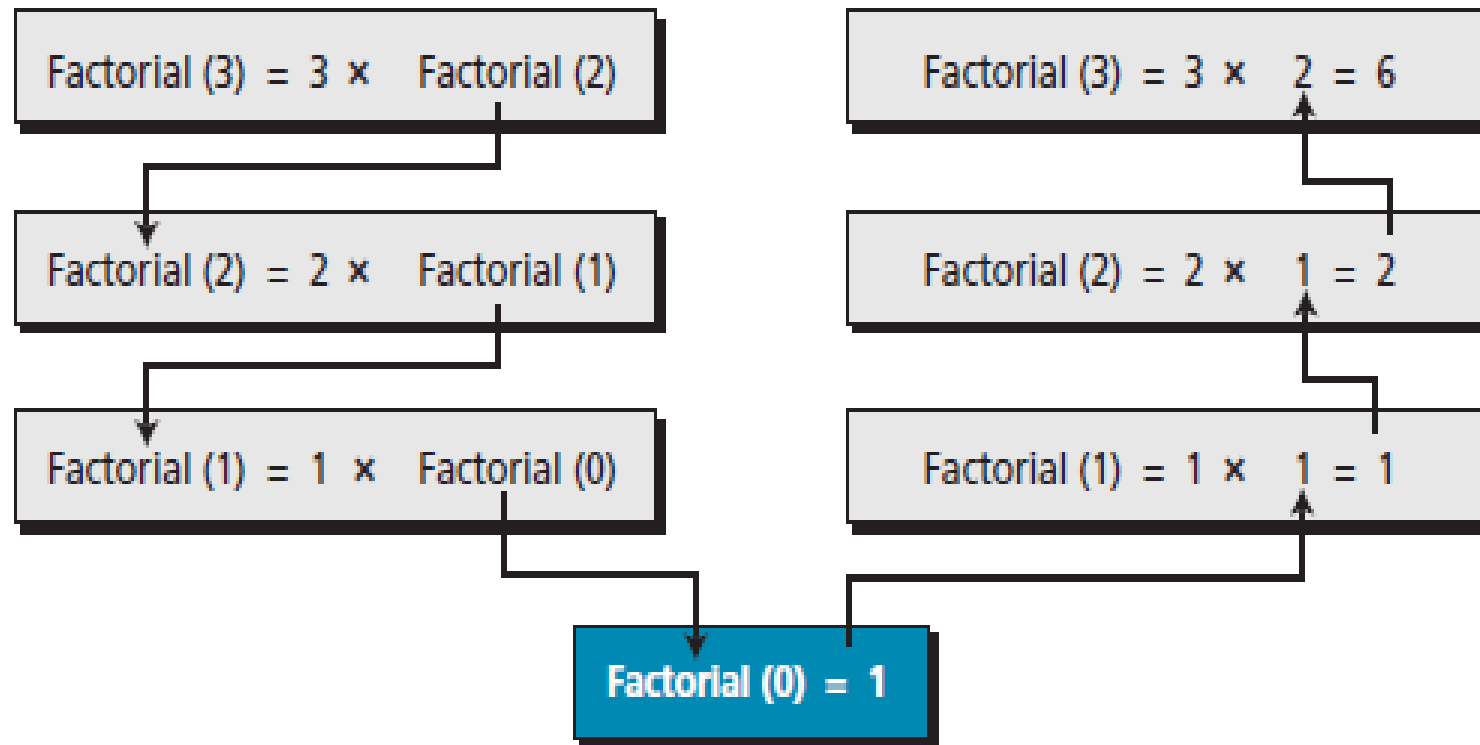
Figure 8.22 *Recursive definition of factorial*

$$\text{Factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{Factorial}(n-1) & \text{if } n > 0 \end{cases}$$

- In general, there are two approaches to writing algorithms for solving a problem. One uses iteration, the other uses recursion.
- Recursion is a process in which an algorithm calls itself.

Tracing the recursive solution to the factorial problem

Figure 8.23 *Tracing the recursive solution to the factorial problem*



Iteration solution the factorial problem

Algorithm 8.6 Iteration solution to the factorial problem

Algorithm: Factorial (n)

Purpose: Find the factorial of a number using a loop

Pre: Given: n

Post: None

Return: $n!$

```
{  
     $F \leftarrow 1$   
     $i \leftarrow 1$   
    while ( $i \leq n$ )  
    {  
         $F \leftarrow F \times i$   
         $i \leftarrow i + 1$   
    }  
    return  $F$   
}
```

A recursive solution to the factorial problem

Algorithm 8.7 A recursive solution of the factorial problem

Algorithm: Factorial (n)

Purpose: Find the factorial of a number using recursion

Pre: Given: n

Post: None

Return: $n!$

```
{  
    if ( $n = 0$ )  
        return 1  
    else  
        return  $n \times \text{Factorial}(n - 1)$   
}
```

Exercises

- P235 8.9.3 Problems
 - p8-5, p8-6, p8-7, p8-17, p8-21, p8-22, p8-23